TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 1: Theoretical Questions

Foreign Trade University

Instructions

Part A contains 22 questions requiring written explanations or short proofs. Part B is a 10-question quiz (true/false and multiple choice). Attempt every question on your own before consulting Part C (Solutions) at the end of this document.

Part A: Theory Questions

A1. Explain the difference between *worst-case*, *average-case*, and *best-case* running time. Why does this course (and most of the algorithms literature) default to worst-case analysis?

A2. Prove that Big-O is transitive: if $f(n) = O(g(n))$ and $g(n) = O(h(n))$, then $f(n) = O(h(n))$.

A3. Order the following functions from slowest- to fastest-growing, and justify the relative order of any two consecutive functions in your list:

$$n^2, \quad 2^n, \quad n \log n, \quad \log n, \quad n^3, \quad \sqrt{n}, \quad n, \quad 1.$$

A4. Asymptotic analysis ignores constant factors and lower-order terms (e.g. $3n^2 + 100n + 5 = \Theta(n^2)$). Give a concrete scenario where this abstraction could be *misleading* in practice – i.e. where an algorithm with a worse asymptotic bound could still be faster for the input sizes that actually occur.

A5. Compare arrays and singly linked lists in terms of (a) random access, (b) insertion at the front, (c) insertion at the end, and (d) memory overhead per element. For each of the four, state which structure is better and why.

A6. Explain why a *stack* (LIFO) is the natural data structure for checking whether brackets in an expression are balanced. What property of nested structures makes LIFO order the right choice (as opposed to FIFO)?

A7. A binary min-heap is stored as an array, where the children of index i are at indices $2i + 1$ and $2i + 2$. Explain why both `push` (insert + sift up) and `pop` (remove min + sift down) run in $O(\log n)$ time, in terms of the height of the tree.

A8. Using your answer to A7, explain why `heapsort` (push all n elements, then pop them all) runs in $O(n \log n)$ time overall.

A9. State the Stable Matching Problem precisely: define a perfect matching, define a *rogue couple* (blocking pair), and define what it means for a matching to be *stable*.

A10. Prove that the (men-proposing) Gale–Shapley algorithm terminates after at most n^2 iterations of its main loop, where $n = |M| = |W|$.

A11. Prove that the matching produced by Gale–Shapley is a *perfect* matching (every man and every woman ends up matched). Your proof should explain why a woman, once engaged, can never become *permanently* free again.

A12. Prove that the matching produced by Gale–Shapley admits *no rogue couples* (i.e. it is stable). Your proof should consider an arbitrary pair (m, w) not in the matching and show it cannot be a rogue couple.

A13. Define *valid partner*. State the Men-Optimality Theorem, and explain in your own words (an informal argument is fine) why the proposing side ends up with their best possible valid partner.

A14. State the Women-Pessimality Theorem. Explain the apparent "unfairness" it describes, and why this is an unavoidable consequence of the Men-Optimality Theorem (not an independent fact that could go either way).

A15. Give a 2×2 instance of the Stable Matching Problem (preference lists for 2 men and 2 women) that has *two* distinct stable matchings. For each of the two matchings, identify which side (men or women) prefers it.

A16. In the Hospital–Residents problem, each hospital h has a capacity capacity$(h) > 1$. Explain how the resident-proposing algorithm generalizes Gale–Shapley, and why the resulting assignment is still "stable" (define stability for this many-to-one setting).

A17. With *incomplete* preference lists, some participants may end up unmatched. Explain how the definition of "rogue couple" must be adjusted to account for the possibility that a participant prefers being unmatched to being matched with someone not on their list.

A18. Karatsuba's algorithm multiplies two n-digit numbers using 3 recursive multiplications of $n/2$-digit numbers, plus $O(n)$ additional work for additions/shifts. Write down the recurrence $T(n) = \ldots$ and solve it (you may use the Master Theorem, or unroll the recursion tree) to show $T(n) = \Theta(n^{\log_2 3})$. Approximately what is $\log_2 3$?

A19. Draw (or describe) the recursion tree for `fib_naive(5)`. Explain why the number of calls grows exponentially with n, and explain precisely what memoization changes about this tree.

A20. Explain the relationship between the number of recursive calls made by `binary_search_recursive` on an array of size n and $\log_2 n$. Why does doubling n only add (roughly) one more call?

A21. Insertion sort makes exactly $n - 1$ comparisons on an already-sorted array but $\frac{n(n-1)}{2}$ on a reverse-sorted array. Explain both bounds, and explain what it means for an algorithm to be "adaptive" (i.e. faster on nearly-sorted input).

A22. *Discussion (open-ended).* In the Gale–Shapley algorithm, the side that proposes ends up better off (Men-Optimality) and the side that receives proposals ends up worse off (Women-Pessimality), *assuming everyone reports their true preferences.* Suppose a woman could submit a *false* preference list. Could she ever obtain a better partner than the one she gets under Gale–Shapley with true preferences (assuming all men still report truthfully)? Discuss informally – you do not need a formal proof, but justify your answer with an example or an argument about how the algorithm uses preference lists.

Part B: Quiz

B1. (True/False) If $f(n) = O(n)$, then $f(n) = \Omega(n)$.

B2. (Short answer) What is the tightest $\Theta(\cdot)$ bound for $f(n) = 5n^2 + 100n + 7$?

B3. (Multiple choice) A binary min-heap with n elements stored in an array has height:

(a) $O(1)$

(b) $O(\log n)$

(c) $O(n)$

(d) $O(n \log n)$

B4. (Short answer) For $n = 4$ men and $n = 4$ women, what is the *maximum possible* number of proposals the men-proposing Gale–Shapley algorithm could make (worst case, using the n^2 bound)?

B5. (Multiple choice) Which data structure is the natural choice for simulating round-robin CPU scheduling?

(a) Stack (LIFO)

(b) Queue (FIFO)

(c) Binary min-heap

(d) Sorted array

B6. (True/False) In Gale–Shapley, once a woman becomes engaged, the desirability (to her) of her partner is non-decreasing over the rest of the algorithm's execution – i.e. she only ever trades up.

B7. (Short answer) Karatsuba's algorithm reduces the number of recursive multiplications needed per level from 4 (grade-school) to how many?

B8. (Short answer) For fib(20) with $F(0) = 0, F(1) = 1$, the naive recursive implementation makes 21891 total calls. Roughly how many calls does the memoized version make (order of magnitude, and exact small multiple of n)?

B9. (True/False) Every instance of the Stable Matching Problem has exactly one stable matching.

B10. (Short answer) For a sorted array of length $n = 15$, what is the maximum number of recursive calls `binary_search_recursive` makes (i.e. $\lceil \log_2(n+1) \rceil$)?

Part C: Solutions

Part A

A1. **Worst-case** running time is the maximum running time over all inputs of a given size n; **average-case** is the expected running time under some distribution over inputs; **best-case** is the minimum. The course defaults to worst-case because (a) it gives a guarantee that holds for *every* input, with no assumptions about the input distribution, and (b) average-case analysis requires committing to a (often unrealistic) probability distribution over inputs, which is harder to justify and to get right.

A2. Since $f(n) = O(g(n))$, there exist c_1, n_1 such that $f(n) \leq c_1 g(n)$ for all $n \geq n_1$. Since $g(n) = O(h(n))$, there exist c_2, n_2 such that $g(n) \leq c_2 h(n)$ for all $n \geq n_2$. Let $n_0 = \max(n_1, n_2)$ and $c = c_1 c_2$. For all $n \geq n_0$, $f(n) \leq c_1 g(n) \leq c_1(c_2 h(n)) = c \cdot h(n)$. Hence $f(n) = O(h(n))$.

A3. From slowest- to fastest-growing:

$$1 \prec \log n \prec \sqrt{n} \prec n \prec n \log n \prec n^2 \prec n^3 \prec 2^n.$$

Justifications for consecutive pairs: $1 \prec \log n$ because $\log n \to \infty$ while 1 is constant. $\log n \prec \sqrt{n}$ because $\log n = o(n^\epsilon)$ for any $\epsilon > 0$. $\sqrt{n} \prec n$ trivially ($\sqrt{n} = n^{1/2}$). $n \prec n \log n$ since $\log n \to \infty$. $n \log n \prec n^2$ since $\log n = o(n)$. $n^2 \prec n^3$ trivially. $n^3 \prec 2^n$ since any polynomial is $o(2^n)$ (exponentials eventually dominate all polynomials).

A4. Example: an $O(n^2)$ algorithm with very small constants (say running time $\approx 2n^2$ microseconds) versus an $O(n \log n)$ algorithm with large constants (say $\approx 1000\, n \log n$ microseconds). For $n = 100$: the "worse" algorithm takes $2 \cdot 100^2 = 20{,}000$ microseconds, while the "better" one takes $1000 \cdot 100 \cdot \log_2 100 \approx 1000 \cdot 100 \cdot 6.6 \approx 664{,}000$ microseconds. For input sizes that stay in this range (e.g. small fixed-size problems solved many times), the asymptotically worse algorithm is faster in practice. This is why real systems often use, e.g., insertion sort for small sub-arrays inside an otherwise $O(n \log n)$ sort.

A5.

Operation	Better structure	Why
(a) Random access	Array	$O(1)$ index arithmetic vs. $O(n)$ pointer walk
(b) Insert at front	Linked list	$O(1)$ new head vs. $O(n)$ shift of all elements
(c) Insert at end	Tie / depends	Array: $O(1)$ amortized (with resizing); linked list: $O(1)$ only with
(d) Memory overhead	Array	No per-element pointer storage, better cache locality

A6. Brackets nest: the bracket that must be closed *next* is always the *most recently opened* one that is still open. This "most recent first" access pattern is exactly LIFO order. Pushing each opening bracket and popping on each closing bracket means the top of the stack is always the innermost unmatched opening bracket – exactly the one the next closing bracket must match. A queue (FIFO) would instead track the *oldest* unmatched bracket, which is the wrong one.

A7. In a binary heap stored as an array, a complete binary tree with n nodes has height $\lfloor \log_2 n \rfloor$ (each level roughly doubles the number of nodes). Both sift-up (after `push`, bubble a node toward the root) and sift-down (after `pop`, bubble a node toward the leaves) move along a single path from one level to an adjacent level, doing $O(1)$ work per level. Since the path length is bounded by the tree height $O(\log n)$, both operations are $O(\log n)$.

A8. `heapsort` performs n `push` operations and n `pop` operations, each $O(\log n)$ by A7. Total time is $n \cdot O(\log n) + n \cdot O(\log n) = O(n \log n)$.

A9. A *perfect matching* S is a set of n pairs (m, w), $m \in M$, $w \in W$, such that every man and every woman appears in exactly one pair (a bijection $M \leftrightarrow W$). A pair $(m, w) \notin S$ is a *rogue couple* (blocking pair) if m prefers w to his partner in S *and* w prefers m to her partner in S. A matching is *stable* if it is a perfect matching with no rogue couples.

A10. Each iteration of the main loop corresponds to one man proposing to one woman he has *never proposed to before* (a man never repeats a proposal to the same woman). There are n men, and each has at most n women on his preference list, so the total number of (man, woman) pairs that could ever be proposed is at most $n \times n = n^2$. Since each loop iteration consumes one such pair and no pair is reused, the loop runs at most n^2 times.

A11. *Key fact:* once a woman becomes engaged, she never becomes free again – she can only *switch* to a more-preferred partner (her old partner becomes free, but she remains engaged, just to someone new). So the set of engaged women is monotonically non-decreasing over time.

Now suppose, for contradiction, that the algorithm terminates with some man m still free. Since the loop only stops when no man is both free *and* has unproposed women left, m must have proposed to (and been rejected by) every one of the n women, meaning all n women are currently engaged – to men other than m (since m is free). But that means at least n men are engaged (one per engaged woman, since engagements are one-to-one) – plus m himself is unengaged, giving at least $n + 1$ men, contradicting $|M| = n$. So no man can be left free at termination; since $|M| = |W| = n$ and the matching is one-to-one, every woman is matched too. Hence the result is a perfect matching.

A12. Take any pair $(m, w) \notin S$, where S is the final matching, with m matched to $w' \neq w$ and w matched to $m' \neq m$. We show (m, w) is not a rogue couple by considering two cases for whether m ever proposed to w:

- m **never proposed to** w**.** Men propose in strictly decreasing order of preference, and m's *last* proposal (the one that succeeded) was to w'. If m never proposed to w, it must be because m prefers w' to w (he would have reached w eventually otherwise, or stopped at w' which he prefers more). So m does *not* prefer w to w' – the first condition for a rogue couple fails.
- m **proposed to** w **at some point, and was rejected (immediately or later displaced).** A woman rejects/displaces a partner only in favor of someone she prefers *more*, and once engaged her partner quality is non-decreasing (as established in A11). So w's final partner m' is someone w likes at least as much as m. Hence w does *not* prefer m to m' – the second condition for a rogue couple fails.

In both cases, (m, w) is not a rogue couple. Since (m, w) was arbitrary, S has no rogue couples, i.e. S is stable.

A13. A woman w is a *valid partner* of man m if there is *some* stable matching (not necessarily the one Gale–Shapley produces) in which m and w are paired. The **Men-Optimality Theorem** states that in the matching S^* produced by the men-proposing Gale–Shapley algorithm, every man m is matched to the *best* (most preferred) woman among all his valid partners.

Informal argument: suppose some man m ended up with a partner w that is *not* his best valid partner – i.e. there is a better valid partner w^* that m prefers, and some stable matching T

where m and w^* are paired. Tracing through the algorithm, m proposes in decreasing order of preference, so he would have proposed to w^* before w. For m to end up with w instead of w^*, m must have been rejected by w^* at some point, which only happens if w^* received a proposal from a man she prefers even more than m. One can show (by a more careful induction over the course of the algorithm, omitted here) that this would force a contradiction with T being stable – intuitively, w^*'s "better" partner in Gale–Shapley would themselves form a rogue couple with someone in T. The full induction is in Kleinberg & Tardos, Theorem 1.7.

A14. The **Women-Pessimality Theorem** states that in the same matching S^*, every woman is matched with her *worst* valid partner (the least-preferred partner she has in any stable matching). This seems unfair to the women, but it is a direct *consequence* of Men-Optimality, not an independent fact: in any *fixed* matching, if man m is matched to woman w, that uses up one "slot". If every man simultaneously gets his *best* valid partner (Men-Optimality), then for any woman w, her partner in S^* must be the *worst* man among those who consider w a valid partner – otherwise some man would have to be matched to a partner worse than his best valid partner, contradicting Men-Optimality for that man. The two theorems are two views of the same underlying structure (the lattice of stable matchings).

A15. Example: men $\{M_1, M_2\}$, women $\{W_1, W_2\}$, where

$$M_1 : W_1 > W_2, \quad M_2 : W_2 > W_1, \quad W_1 : M_2 > M_1, \quad W_2 : M_1 > M_2 \quad \text{(a "rotation")}.$$

Both $S_A = \{(M_1, W_1), (M_2, W_2)\}$ and $S_B = \{(M_1, W_2), (M_2, W_1)\}$ are stable (in each, every man is paired with his *first*-choice woman in S_A, or every woman is paired with her first-choice man in S_B – check there are no rogue couples in either). S_A is the men-optimal (and women-pessimal) matching; S_B is the women-optimal (and men-pessimal) matching. So the *men* prefer S_A and the *women* prefer S_B.

A16. Each resident proposes down their preference list, one hospital at a time, exactly as in the one-to-one case. The difference is that a hospital h does not simply hold "the best resident seen so far" – it holds up to `capacity`(h) residents at once. When a new resident proposes, if h has fewer than `capacity`(h) residents currently held, it accepts; otherwise it compares the new resident to its *currently-held resident it likes least*, and keeps whichever it prefers, rejecting the other. **Stability** here means: no resident r and hospital h (with r not assigned to h) such that r prefers h to their current assignment *and* h either has a free slot or prefers r to one of its currently-assigned residents. The proof of termination/stability is essentially the same as the one-to-one case, applied "per slot".

A17. With incomplete lists, a pair (m, w), both currently unmatched (or each preferring to be unmatched than matched to their current partner, if any), where *both* m and w find each other acceptable (each appears on the other's list), would form a rogue couple if m and w both prefer being matched to each other over their current situation (including the possibility of being unmatched). Formally, "m prefers w to his current partner" must be extended to also cover the case where m's "current partner" is *being unmatched* – and by the rules of the algorithm, being matched to anyone on your list is always preferred to being unmatched, so this only matters for checking that no acceptable pair of currently-unmatched (or mutually-improvable) participants exists.

A18. The recurrence is

$$T(n) = 3\,T(n/2) + O(n).$$

By the Master Theorem (case where $a = 3$, $b = 2$, and $f(n) = O(n) = O(n^{\log_2 3 - \epsilon})$ is asymptotically smaller than $n^{\log_b a} = n^{\log_2 3}$), the recursive term dominates and $T(n) = \Theta(n^{\log_2 3})$. Numerically, $\log_2 3 \approx 1.585$, so $T(n) = \Theta(n^{1.585})$, which is asymptotically better than the grade-school $\Theta(n^2)$.

(Alternative derivation by unrolling: at depth k, there are 3^k sub-problems each of size $n/2^k$, contributing $O(3^k \cdot n/2^k)$ work at that level. Summing over $k = 0, \ldots, \log_2 n$ gives a geometric series dominated by its last term, $3^{\log_2 n} = n^{\log_2 3}$.)

A19. `fib_naive(5)` calls `fib_naive(4)` and `fib_naive(3)`; `fib_naive(4)` calls `fib_naive(3)` and `fib_naive(2)`; and so on. Crucially, `fib_naive(3)` is computed *twice* (once as a child of `fib_naive(5)`'s call to `fib_naive(4)`, and once directly), and this duplication compounds at every level – the number of calls to compute $F(n)$ satisfies roughly the same recurrence as $F(n)$ itself, i.e. $C(n) = C(n-1) + C(n-2) + O(1)$, which grows like ϕ^n (the golden ratio to the n), i.e. exponentially.

Memoization caches the result of $F(k)$ the first time it's computed. Every subsequent call to `fib_memo`(k) for the *same* k becomes an $O(1)$ cache lookup instead of a recursive re-computation. Since there are only $n + 1$ distinct sub-problems $(F(0), \ldots, F(n))$, and each is computed exactly once (doing $O(1)$ work beyond its two recursive sub-calls, which are now cache hits), the total work collapses from exponential to $O(n)$.

A20. Each recursive call to `binary_search_recursive` discards *half* of the remaining search range. Starting from n elements, after k calls the range has size roughly $n/2^k$. The recursion stops (range becomes empty) when $n/2^k < 1$, i.e. $k > \log_2 n$. So the number of calls is $\Theta(\log_2 n)$, specifically $\lceil \log_2(n+1) \rceil$. Doubling n adds exactly one more halving step needed to reach an empty range, hence "roughly one more call" – this is the defining characteristic of logarithmic growth.

A21. On an *already-sorted* array, when inserting the i-th element (0-indexed), it is compared only once with its immediate predecessor and found to be already in the correct position – so each of the $n - 1$ insertions (for $i = 1, \ldots, n-1$) costs exactly 1 comparison, for a total of $n - 1$. On a *reverse-sorted* array, the i-th element (0-indexed) must be compared with *all* i previously-placed elements before finding its position (at the very front), so the total is $1 + 2 + \cdots + (n-1) = \frac{n(n-1)}{2}$.

An algorithm is **adaptive** if its running time improves on "nearly sorted" input – insertion sort is adaptive because the number of comparisons is proportional to the number of *inversions* (pairs of elements out of order) in the input, which is small for nearly-sorted arrays and large ($\Theta(n^2)$) for reverse-sorted arrays. Many classic $O(n \log n)$ algorithms (e.g. standard mergesort) are *not* adaptive in this sense – they take $\Theta(n \log n)$ regardless of the input order.

A22. This is a discussion question, but here is the expected line of reasoning. In the men-proposing algorithm, women never *act* – they only passively accept or reject based on their stated preferences. A woman's outcome is entirely determined by (i) the true preferences of the men (which determine who proposes to her and in what order), and (ii) her own stated preference list (which determines which of those proposals she accepts). It is a known result (related to the Gale–Shapley paper and later work by Dubins and Freedman, and Roth) that the men-proposing Gale–Shapley mechanism is **strategy-proof for the proposing side** (men can never benefit from lying) but **not strategy-proof for the receiving side** (a woman can sometimes obtain a better outcome by mis-stating her preferences – e.g. by rejecting

a proposal from a man she actually prefers to her eventual Gale–Shapley partner, hoping a better one arrives later). However, this requires the woman to have information about other participants' preferences, and is a delicate, risky strategy if her beliefs are wrong (she might end up unmatched or with someone worse). The key takeaway: *truthfulness is not symmetric* – which side benefits from the mechanism's incentive structure depends on which side proposes.

Part B

B1. **False.** $O(n)$ is an *upper* bound (grows no faster than n); $\Omega(n)$ is a *lower* bound (grows at least as fast as n). $f(n) = O(n)$ alone does not guarantee f grows *at least* linearly – e.g. $f(n) = 1$ satisfies $f(n) = O(n)$ but $f(n) \neq \Omega(n)$.

B2. $\Theta(n^2)$. The lower-order terms ($100n$, 7) and the constant factor (5) do not affect the asymptotic growth class.

B3. **(b)** $O(\log n)$**.** A complete binary tree with n nodes has height $\lfloor \log_2 n \rfloor$.

B4. $n^2 = 4^2 = 16$ proposals (the bound is n^2, achieved when $n = 4$ gives at most 16).

B5. **(b) Queue (FIFO).** Round-robin processes tasks in arrival order, cycling each task to the back of the line when its time slice expires – exactly FIFO behaviour.

B6. **True.** Once engaged, a woman only changes partners when a man she prefers *more* than her current partner proposes – so her partner's rank (in her own preference list) only improves (or stays the same) over time.

B7. **3.** Karatsuba uses the identity $x_1y_0 + x_0y_1 = (x_1 + x_0)(y_1 + y_0) - x_1y_1 - x_0y_0$ to compute the cross term from the two "corner" products plus one extra product, for 3 total recursive multiplications instead of 4.

B8. $O(n)$, specifically 39 calls for $n = 20$ – roughly $2n$ (each $F(k)$ for $k = 0, \ldots, 20$ is computed once, plus a small constant number of bookkeeping calls).

B9. **False.** As shown in A15, an instance can have multiple stable matchings (e.g. both the men-optimal and women-optimal matchings, when they differ).

B10. $\lceil \log_2(16) \rceil = 4$. (Since $n = 15$, $n + 1 = 16 = 2^4$, so $\lceil \log_2 16 \rceil = 4$.)